

INFLUENCELIFT AI

Coding Interview Workbook

Forty project-grounded coding and debugging exercises, with solution outlines and senior-level extensions.

Portfolio Edition 1.0

Primary author	Mohit Bhatnagar
Collaboration credit	Aarushi Rai - marketing analytics framing and portfolio positioning
Live application	https://influencelift-ai.streamlit.app
Source repository	https://github.com/mohit231007/influencelift-ai

Purpose: help the project owner explain, defend, extend, and market the system in technical and business interviews.

How to use this guide

Use the first ten exercises for implementation fluency, the next twenty for ML/API product coding, and the final ten for quality, deployment, and system debugging.

This is a practical interview and portfolio manual. Each numbered module is intentionally self-contained so you can revise one topic, rehearse a two-minute answer, and then practise a deeper follow-up.

Module	Topic
01	Write a robust human-number parser
02	Validate a raw campaign schema
03	Repair follower scale anomalies
04	Audit negative spend without hiding errors
05	Build a scikit-learn transformer
06	Prevent preprocessing leakage
07	Implement median imputation in a pipeline
08	Tune Ridge with cross-validation
09	Compare Ridge and XGBoost fairly
10	Generate out-of-fold predictions
11	Calculate RMSE, MAE, and R^2
12	Build an empirical prediction interval
13	Detect univariate extrapolation
14	Design a multivariate OOD check
15	Serialize a model bundle
16	Version a prediction artefact
17	Generate deterministic synthetic data
18	Write a CLI training command
19	Write a CLI prediction command
20	Design a FastAPI request model
21	Implement a single prediction endpoint
22	Implement a batch endpoint
23	Return structured validation errors
24	Add a health endpoint
25	Cache a model in Streamlit
26	Create a messy-input prediction form
27	Build a batch upload workflow
28	Create a downloadable result
29	Implement scenario comparison
30	Calculate incremental cost per sale
31	Test parser edge cases
32	Test a custom transformer
33	Test API happy and failure paths
34	Write a GitHub Actions matrix
35	Create a service smoke test
36	Harden a Dockerfile
37	Add structured logging
38	Design model monitoring tables
39	Refactor row-wise audit for scale
40	Complete a full system debugging exercise

01. Write a robust human-number parser

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **write a robust human-number parser** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_01(input_data, config=None):  
    """Write a robust human-number parser."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for write a robust human-number parser starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

02. Validate a raw campaign schema

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **validate a raw campaign schema** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_02(input_data, config=None):  
    """Validate a raw campaign schema."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for validate a raw campaign schema starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

03. Repair follower scale anomalies

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **repair follower scale anomalies** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_03(input_data, config=None):  
    """Repair follower scale anomalies."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for repair follower scale anomalies starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

04. Audit negative spend without hiding errors

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **audit negative spend without hiding errors** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_04(input_data, config=None):
    """Audit negative spend without hiding errors."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for audit negative spend without hiding errors starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

05. Build a scikit-learn transformer

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **build a scikit-learn transformer** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_05(input_data, config=None):  
    """Build a scikit-learn transformer."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for build a scikit-learn transformer starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

06. Prevent preprocessing leakage

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for ****prevent preprocessing leakage**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_06(input_data, config=None):  
    """Prevent preprocessing leakage."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for prevent preprocessing leakage starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

07. Implement median imputation in a pipeline

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for ****implement median imputation in a pipeline**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_07(input_data, config=None):
    """Implement median imputation in a pipeline."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for implement median imputation in a pipeline starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

08. Tune Ridge with cross-validation

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **tune ridge with cross-validation** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_08(input_data, config=None):
    """Tune Ridge with cross-validation."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for tune ridge with cross-validation starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

09. Compare Ridge and XGBoost fairly

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for **compare ridge and xgboost fairly** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_09(input_data, config=None):
    """Compare Ridge and XGBoost fairly."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for compare ridge and xgboost fairly starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

10. Generate out-of-fold predictions

Difficulty: Foundation

Interview prompt

Implement or explain a production-quality solution for ****generate out-of-fold predictions**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_10(input_data, config=None):  
    """Generate out-of-fold predictions."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for generate out-of-fold predictions starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

11. Calculate RMSE, MAE, and R^2

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **calculate rmse, mae, and r^2** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_11(input_data, config=None):
    """Calculate RMSE, MAE, and  $R^2$ ."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for calculate rmse, mae, and r^2 starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

12. Build an empirical prediction interval

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **build an empirical prediction interval** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_12(input_data, config=None):  
    """Build an empirical prediction interval."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for build an empirical prediction interval starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

13. Detect univariate extrapolation

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **detect univariate extrapolation** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_13(input_data, config=None):  
    """Detect univariate extrapolation."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for detect univariate extrapolation starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

14. Design a multivariate OOD check

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **design a multivariate ood check** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_14(input_data, config=None):  
    """Design a multivariate OOD check."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for design a multivariate ood check starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

15. Serialize a model bundle

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **serialize a model bundle** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_15(input_data, config=None):  
    """Serialize a model bundle."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for serialize a model bundle starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

16. Version a prediction artefact

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **version a prediction artefact** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_16(input_data, config=None):  
    """Version a prediction artefact."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for version a prediction artefact starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

17. Generate deterministic synthetic data

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **generate deterministic synthetic data** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_17(input_data, config=None):
    """Generate deterministic synthetic data."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for generate deterministic synthetic data starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

18. Write a CLI training command

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for ****write a cli training command**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_18(input_data, config=None):
    """Write a CLI training command."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for write a cli training command starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

19. Write a CLI prediction command

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for ****write a cli prediction command**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_19(input_data, config=None):  
    """Write a CLI prediction command."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for write a cli prediction command starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

20. Design a FastAPI request model

Difficulty: Intermediate

Interview prompt

Implement or explain a production-quality solution for **design a fastapi request model** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_20(input_data, config=None):
    """Design a FastAPI request model."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for design a fastapi request model starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

21. Implement a single prediction endpoint

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **implement a single prediction endpoint** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_21(input_data, config=None):  
    """Implement a single prediction endpoint."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for implement a single prediction endpoint starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

22. Implement a batch endpoint

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **implement a batch endpoint** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_22(input_data, config=None):  
    """Implement a batch endpoint."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for implement a batch endpoint starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

23. Return structured validation errors

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **return structured validation errors** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_23(input_data, config=None):
    """Return structured validation errors."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for return structured validation errors starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

24. Add a health endpoint

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for ****add a health endpoint**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_24(input_data, config=None):  
    """Add a health endpoint."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for add a health endpoint starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

25. Cache a model in Streamlit

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **cache a model in streamlit** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_25(input_data, config=None):  
    """Cache a model in Streamlit."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for cache a model in streamlit starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

26. Create a messy-input prediction form

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **create a messy-input prediction form** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_26(input_data, config=None):
    """Create a messy-input prediction form."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for create a messy-input prediction form starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

27. Build a batch upload workflow

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **build a batch upload workflow** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_27(input_data, config=None):  
    """Build a batch upload workflow."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for build a batch upload workflow starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

28. Create a downloadable result

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **create a downloadable result** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_28(input_data, config=None):  
    """Create a downloadable result."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for create a downloadable result starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

29. Implement scenario comparison

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **implement scenario comparison** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_29(input_data, config=None):  
    """Implement scenario comparison."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for implement scenario comparison starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

30. Calculate incremental cost per sale

Difficulty: Advanced

Interview prompt

Implement or explain a production-quality solution for **calculate incremental cost per sale** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_30(input_data, config=None):  
    """Calculate incremental cost per sale."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for calculate incremental cost per sale starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

31. Test parser edge cases

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **test parser edge cases** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_31(input_data, config=None):  
    """Test parser edge cases."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for test parser edge cases starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

32. Test a custom transformer

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **test a custom transformer** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_32(input_data, config=None):  
    """Test a custom transformer."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for test a custom transformer starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

33. Test API happy and failure paths

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **test api happy and failure paths** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_33(input_data, config=None):  
    """Test API happy and failure paths."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for test api happy and failure paths starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

34. Write a GitHub Actions matrix

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for ****write a github actions matrix**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_34(input_data, config=None):  
    """Write a GitHub Actions matrix."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for write a github actions matrix starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

35. Create a service smoke test

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **create a service smoke test** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_35(input_data, config=None):  
    """Create a service smoke test."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for create a service smoke test starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

36. Harden a Dockerfile

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **harden a dockerfile** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_36(input_data, config=None):  
    """Harden a Dockerfile."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for **harden a dockerfile** starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

37. Add structured logging

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for ****add structured logging**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_37(input_data, config=None):  
    """Add structured logging."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for add structured logging starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

38. Design model monitoring tables

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **design model monitoring tables** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.
- Include at least one normal, boundary, and failure test.

```
def exercise_38(input_data, config=None):  
    """Design model monitoring tables."""  
    # 1. validate contract  
    # 2. implement the smallest correct behaviour  
    # 3. return result plus useful metadata  
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for design model monitoring tables starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

39. Refactor row-wise audit for scale

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for ****refactor row-wise audit for scale**** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.

```
def exercise_39(input_data, config=None):
    """Refactor row-wise audit for scale."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for refactor row-wise audit for scale starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.

40. Complete a full system debugging exercise

Difficulty: Senior design

Interview prompt

Implement or explain a production-quality solution for **complete a full system debugging exercise** in the context of InfluenceLift AI. Your solution must be deterministic, typed where practical, easy to test, and consistent between training and inference.

Constraints and acceptance criteria

- Do not silently discard invalid campaign records.
- Preserve row order and campaign identifiers.
- Return actionable error or audit information.
- Avoid fitting preprocessing on validation/test data.

```
def exercise_40(input_data, config=None):
    """Complete a full system debugging exercise."""
    # 1. validate contract
    # 2. implement the smallest correct behaviour
    # 3. return result plus useful metadata
    raise NotImplementedError
```

Solution approach

- Clarify input and output contracts before selecting libraries.
- Separate pure transformation logic from I/O or framework code.
- Make edge-case policy explicit through configuration or named constants.
- Write tests first for the corruption or failure that motivated the feature.
- Measure complexity and identify the first bottleneck at 100x scale.

Reference answer outline

A strong answer for complete a full system debugging exercise starts with the data contract, implements a narrow reusable function, and then integrates it through the shared package. The candidate should explain why the code avoids training-serving skew, how failure information reaches the user, and which test proves the behaviour. Extra credit comes from discussing complexity, observability, and a safer alternative when assumptions do not hold.

Extension: Change one assumption - file size, feature count, multi-tenancy, or strict latency - and redesign the solution without rewriting unrelated components.